Normalizing DNP3 Response Timing and Segment Shape in the Network to Resist Outstation Fingerprinting

Author Name(s)
Affiliation
Email address

***Abstract*—An adversary preparing an attack on a power-grid control system first fingerprints its field devices, and it can do so from unencrypted DNP3 traffic without decoding the payload, using two metadata features: the cross-layer response time, the gap between a device's TCP acknowledgment and its DNP3 application response, and the shape of the TCP segments the response is delivered in. We present an in-network defense that normalizes both features in the data plane of one programmable switch at the outstation edge, without changing the endpoints or a DNP3 byte. The defense holds a device's acknowledgment and response and releases them at fixed offsets from the request, so the response time becomes a single policy value; it replicates the response and carves it at a boundary DNP3 already places between CRC-protected blocks, turning the native single 49-byte segment into a fixed $[28, 21]$ two-segment shape that the master reassembles as a valid 49-byte response; and it holds a control command and pins the master-visible acknowledgment and echo to the request's arrival, so the observable timing does not vary with a hidden internal release delay. We implement the design as one P4 program on a single Tofino-1 pipeline and evaluate it on a physical SEL-751 relay. On the master-facing link the defended response time is a fixed 4.001 ms for read and select in place of the native 1.3 and 2.1 ms; every one of 1,280 defended responses presents the $[28, 21]$ shape with a CRC- and checksum-valid reconstruction; the control-command gap stays at 4.00 ms across the evaluated delays; and a read-versus-select timing classifier drops to the chance baseline. The evaluation is bounded to one outstation and one response class, and the relay-facing single release is modeled, not measured.**

***Index Terms*—DNP3, SCADA, traffic obfuscation, device fingerprinting, programmable switches, industrial control systems.**

I. Introduction

The grid runs on old protocols, and an attacker learns them before it breaks them. Power utilities move measurements and control commands over protocols such as DNP3 (Distributed Network Protocol 3) [1], which, like most of its peers, ships with no encryption and no authentication by default. Anyone on the path can read the exchange. The cost of that exposure is not hypothetical: in December 2015 remote intruders reached into a Ukrainian power grid and left 225,000 residents in the dark [2]. An attack at that scale does not begin with the attack. It begins with watching, with learning which devices are on the wire and how each behaves, because a control-altering step such as a false-data-injection attack on state estimation only works once the attacker knows the target [3], [4]. What a quiet observer can pull out of unencrypted control traffic is therefore worth taking seriously.

Reading the payload is not even necessary. Traffic-analysis work has shown that packet sizes and timing on their own recover the content of even encrypted sessions [5], [6], and that a device or a device type can be told apart from its wire-side behavior alone [7], [8]. A useful distinction runs through this line of work: an observer may seek a device's *identity*, its *type*, or the *class of transaction* it is currently running. Our concern is the last two: features that let an observer separate one device's transaction classes and, across devices, its type, without decoding anything.

Two features carry that signature for DNP3, and both are visible on unencrypted traffic even though the payload is fully parseable. The first is timing. Formby et al. named the cross-layer response time (CLRT) as a device fingerprint for industrial control systems [9]: the gap between the transport-layer acknowledgment and the application-layer response reflects the device's internal processing and stays stable across sessions. The second is shape. A DNP3 response is delivered in TCP segments, and the vector of segment sizes an observer records is itself a fingerprint, because the outstation lays out its frames deterministically and so produces the same shape every time.

The standard answer to such leakage is traffic obfuscation, and it is well developed for general internet traffic. The goal is to make the observed traffic independent of the real traffic: padding hides a flow's size by adding bytes; morphing transforms one class's size distribution into another's [10]; adaptive padding inserts dummy packets to break the inter-packet timing pattern [11]; and constant-rate and burst-molding schemes force a fixed rate or a colliding shape at a bounded cost [12], [13]. The state of the art moves this into the network so that end hosts need not cooperate. Ditto is the clearest example: running on a programmable switch, it pads every packet and injects chaff packets so that the size, timing, and volume the switch emits follow a fixed template that is independent of the payload, at line rate and without any change to the end hosts [14], and related systems obfuscate flows against traffic analysis in the data plane in the same spirit [15]–[17].

This approach does not fit ICS traffic such as DNP3, for a reason that is structural rather than incidental. General obfuscation is free to *add and fabricate* bytes and packets, and

it targets an *opaque* flow whose only observable is a coarse volume or size pattern. DNP3 violates both premises. Its payload is a validated protocol message: the master reassembles a specific DNP3 response and checks a cyclic redundancy check (CRC) on every 16-byte block, so padding bytes or injected chaff break the exchange the receiver expects. Its endpoints are fixed vendor firmware an operator cannot change. And the leak is not a coarse volume pattern but two precise features, a cross-layer timing interval and a per-response segment shape, that volume shaping does not target. A control command adds a further problem that chaff cannot mask: an outbound OPERATE and its confirmation form a causal pair whose delay is itself an operation-timing fingerprint. A defense for this setting therefore needs a different mechanism, one that normalizes the specific DNP3 timing and shape features while preserving a valid, ordered, CRC-correct DNP3 exchange and changing no endpoint.

Consider one concrete read on the relay we evaluate. The master sends a READ request; the outstation returns first a TCP segment carrying the acknowledgment flag and then, a moment later, its DNP3 application response. The CLRT this paper normalizes is the interval between exactly those two packets: the first acknowledgment-bearing segment and the first DNP3 application response (function 0x81). On our SEL-751 relay that gap has a median near 1.3 ms for a read and near 2.1 ms for a control select, and the response is a single 49-byte segment, a shape we write $[49]$. Those two numbers alone separate the read and select transaction classes, which is the transaction-class leak Formby's fingerprint predicts for this device.

Two further requirements narrow the design. Both features must be handled together, because size-only shaping would leave the timing feature and timing-only shaping would leave the shape feature. And the retiming must not come from a controller on a side path, which would add its own timing variability and a deployment dependency. Given that the defense may not touch the endpoints or the bytes, the one place that satisfies these requirements is a programmable switch on the path, which can retime and re-segment the traffic in the data plane while leaving both endpoints and every DNP3 byte untouched.

We place the defense in the data plane of one programmable switch at the outstation's edge. It does two things. For a response, it holds the acknowledgment and the response and releases each at a fixed offset from the request, so the master-visible CLRT becomes one policy value; and it splits the 49-byte response into a fixed two-segment shape, $[28, 21]$, cutting only at a boundary the protocol already places between CRC-protected blocks, so no DNP3 byte changes and the master reassembles a valid 49-byte response. A control command needs a second treatment, because an outbound OPERATE creates a different causal problem: the delay from the OPERATE to its confirmation is itself an operation-timing fingerprint, and simply releasing the OPERATE on a fixed schedule would still expose it. The defense instead holds the OPERATE and releases it once at an internal delay drawn from a bounded set, while pinning the master-visible acknowledgment and echo to the request's arrival, so the gap the observer measures does not vary with that delay.

Building this on real hardware raised systems problems that shaped the design. The switch has no general-purpose timer, so a delayed release must be realized from packet scheduling. Several events must be anchored to a single request timestamp rather than to whatever arrives next. A valid response must be split without any oracle that would let us claim byte identity to a source frame. The two treatments cannot share one priority schedule without the response's queues starving the command's, so they must run on separate internal scheduling lanes. The whole program must fit one switch pipeline. And the mechanism must operate safely on a physical protective relay.

We evaluate the design on a testbed with a physical master, one Tofino-1 switch, and a physical SEL-751 relay, comparing native and defended traffic captured on the same loaded program under a runtime mode change. On the master-facing link the defended CLRT is a fixed 4.001 ms for both read and select in place of the native 1.3 and 2.1 ms; every one of 1,280 defended eligible responses presents the $[28, 21]$ shape with a sequence-contiguous, CRC-valid, checksum-valid 49-byte reconstruction and no unsplit escape; the control-command gap stays at 4.00 ms across the evaluated internal delays; and a read-versus-select timing classifier drops from 0.59 to the 0.50 chance baseline. The evaluation is bounded: we test one outstation and one eligible response class, so we replace that device's evaluated signature rather than establishing indistinguishability among devices, and we do not observe the relay-facing link, so the single internal release is verified by the implementation model and not measured on the wire. This paper makes the following contributions.

- **A statement of the DNP3 timing-and-shape leak on real hardware.** We measure the native cross-layer response time and segment shape of an SEL-751 relay and show that they separate its read and select transaction classes.
- **An in-network timing normalization.** We hold the acknowledgment and the response in switch queues and release them at fixed offsets from the request, driving the master-visible CLRT to a single policy value without a controller or an endpoint change.
- **A byte-preserving shape normalization.** We replicate the eligible response and carve it at an existing DNP3 CRC-block boundary into a fixed $[28, 21]$ segment shape, so the master reassembles a valid 49-byte response; we claim a CRC- and checksum-valid reconstruction, not byte identity to a source frame.
- **A control-command treatment and its scheduling.** We hold the OPERATE and pin the master-visible acknowledgment and echo to the request arrival, and we show why the two treatments require two independent switch scheduling lanes.
- **A one-program implementation and a bounded hardware evaluation.** We implement the design as one P4

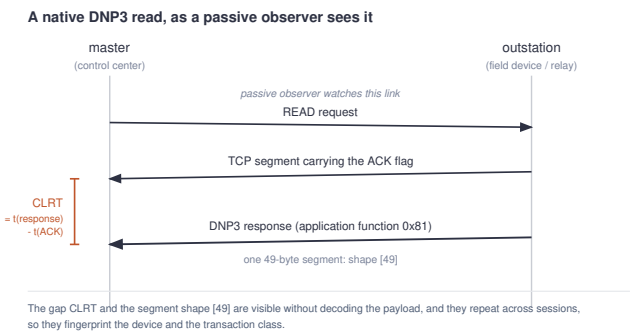

Fig. 1. A native read as a passive observer sees it. After the request, the outstation sends a TCP segment carrying the ACK flag and then the DNP3 application response (function 0x81). The cross-layer response time (CLRT) is the interval between those two packets; the response is a single 49-byte segment, a shape we write $[49]$.

program on a single switch pipeline and evaluate it on a physical SEL-751 with a reproducible capture-to-claim analysis, an explicit safety result, and a stated evidence boundary.

II. Background and Design Overview

This section teaches the two DNP3 transactions the paper works with, the point from which an adversary observes them, the two features that leak, and the shape of the defended traffic we want to produce. It uses no switch-internal names; those appear only in the implementation (Section IV).

A. The native read transaction

A DNP3 exchange runs between a *master* (a control center) and an *outstation* (a field device; here a protective relay). To read data, the master sends a READ request, and the outstation answers. On the wire the answer arrives as two events, shown in Fig. 1: first a TCP segment that carries the acknowledgment (ACK) flag, then the DNP3 application response itself, which DNP3 marks with application function code 0x81. On the SEL-751 the response is a 49-byte DNP3 frame that fits in one TCP segment.

B. The native control transaction

Control is a two-step exchange called select-before-operate. The master first sends a SELECT that arms a control point; the outstation confirms it. The master then sends an OPERATE that actuates the point, and the outstation echoes the command back. The two steps exist so that a stray or replayed OPERATE with no matching SELECT cannot act.

C. The observation point

We assume a passive observer on the *master-facing* link, the segment between the master and the switch. It records TCP and IP headers, TCP segment sizes, and arrival times. It does not see the *relay-facing* link between the switch and the outstation, nor anything internal to the switch. Fig. 2 places the observer and marks the relay-facing link as unobserved.

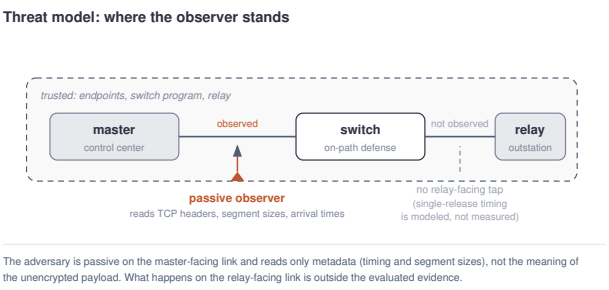

Fig. 2. Threat model. The passive observer sits on the master-facing link and reads only metadata (TCP headers, segment sizes, arrival times). The relay-facing link is not observed, so the internal release timing of a control command is modeled, not measured.

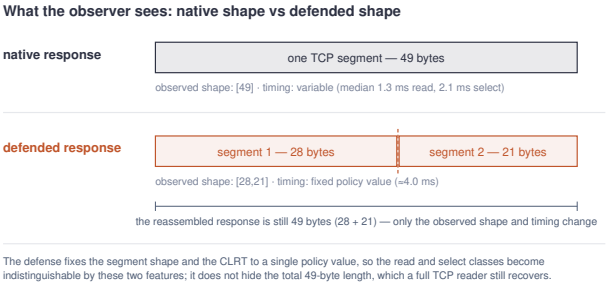

Fig. 3. Native versus defended, as observed. The native response is one 49-byte segment with variable timing; the defended response is the same 49 bytes as a fixed $[28, 21]$ two-segment shape with a fixed CLRT. The total length is unchanged.

D. The two leaked features

From that vantage the observer reads two features without decoding the payload. The *cross-layer response time* is the gap between the ACK-bearing segment and the DNP3 response defined above; it reflects the outstation's internal processing. The *segment shape* is the vector of TCP segment sizes the response is delivered in ($[49]$ natively). Because the outstation is deterministic, both features repeat across sessions, so they fingerprint the device and separate its transaction classes: on the SEL-751 the read and select classes differ in CLRT (medians near 1.3 and 2.1 ms).

E. What defended traffic should look like

The defense produces two changes on the master-facing link, shown in Fig. 3. It fixes the CLRT to a single policy value, so the timing no longer separates the classes. And it fixes the segment shape to $[28, 21]$: the same 49 bytes, now delivered as two segments of 28 and 21 bytes. The cut is placed at a boundary DNP3 already uses. A DNP3 frame is a link header followed by 16-byte data blocks, each closed by a 2-byte CRC that the master validates; cutting between two such blocks yields two segments that each end on an already-valid CRC, so no byte is edited and no CRC is recomputed. Byte 28 is one such boundary. The reassembled response is still 49 bytes: the defense normalizes the observed shape and timing, not the total length, which a full TCP reader still recovers.

F. The two treatments and why they are separated

Two treatments produce these changes. *Response shaping* handles a read or select response: it holds the acknowledgment and the response, releases each at a fixed offset from the request so the CLRT is a constant, and splits the released response into the $[28, 21]$ shape. *The control-command treatment* handles the OPERATE: it holds the command and releases it once at an internal delay, while pinning the master-visible acknowledgment and echo to the request's arrival so that the observable gap does not vary with the internal delay. The two treatments cannot share one priority schedule. Response shaping keeps a set of higher-priority "blocker" packets draining to time its release; if the held control command sat below them on the same schedule, those blockers would starve it and its release would drift toward the response's release time instead of its own. The design therefore gives the two treatments *two independent scheduling lanes* inside the same switch, as Fig. 6 shows; both are internal paths in one switch and one program, not extra devices or a second pipeline.

III. Threat Model, Assumptions, and Goals

A. System and adversary

The system is the master, the switch, and the SEL-751 outstation of Section II, exchanging the native read and control transactions defined there. The adversary is a *passive* observer on the master-facing link: it captures traffic from a mirrored port or a tap and does not modify, inject, or block a packet. Its goal is reconnaissance, to fingerprint the outstation and its transactions from what the traffic exposes.

B. What the adversary can and cannot read

The adversary reads metadata: TCP and IP headers, TCP segment sizes, and arrival times. The traffic is unencrypted, so the DNP3 payload is in fact parseable; our threat, however, uses only the two metadata features of Section II (CLRT and segment shape) and does not depend on application semantics. Following Formby et al. [9], we distinguish three fingerprinting aims: device identity, device type, and transaction-class inference. Our evaluation concerns transaction-class inference (separating read from select by CLRT) on one device, and the replacement of that device's evaluated features; we do not claim device-identity results or indistinguishability among devices. The adversary knows the protocol and may know the defense policy; its advantage comes only from observing the features.

C. Trusted computing base and deployment assumptions

The master, the switch and its loaded program, and the outstation are trusted. The defense runs entirely in the switch data plane, with no host agent, controller fast path, or endpoint change. Two deployment assumptions bound the timing argument. First, the protected flow carries no TCP timestamp option, which would be an independent timing channel; in our evaluation the option was disabled on the host and confirmed absent at the packet level in a preflight check, and the switch program also parses the option so the timing argument is asserted only when it is absent. Second, only the eligible response class (here the 49-byte read and control responses) is shaped; other traffic is forwarded unchanged.

D. Security goals

We state the security goals as testable properties of the master-facing observation. **G1:** the defended CLRT is a single policy value independent of the transaction class. **G2:** every eligible response presents the fixed segment shape $[28, 21]$. **G3:** the master reassembles a sequence-contiguous, CRC-valid, checksum-valid 49-byte response. **G4:** for a control command, the master-visible acknowledgment and echo placement does not vary with the internal release delay. **G5 (safety):** the defense actuates no physical output. A separate evaluation-validity property, that native and defended traffic differ only by a runtime mode on one loaded program, lets the comparison isolate the defense; we treat it as a methodology control rather than a security goal.

E. Non-goals and evidence limitations

The following are out of scope, and the evidence does not establish them. We do not verify the relay-facing link: the internal release time and the number of copies delivered to the relay are not captured, so the single release is a modeled, not a measured, property, and we do not claim exactly-once delivery. We do not hide the total response length, which stays 49 bytes and is recoverable by a full TCP reader. We do not claim byte identity between an emitted response and a source frame, because no paired source-side oracle was captured. We do not generalize from one SEL-751 and one eligible class to all DNP3 outstations, and we do not claim indistinguishability among devices or resistance to every traffic-analysis feature. We do not defend against an active adversary or add payload confidentiality. If the observer could also see the relay-facing link, it could time the internal release directly, and the control-command timing property would no longer hold; the defense is scoped to the master-facing observer.

IV. Implementation

We implement the design of Section II as one P4 program on a single Intel Tofino-1 switch, with a control-plane configuration layer and a guarded control-test harness. This section starts with an end-to-end walkthrough, then details each mechanism. For each mechanism we give the same account: the problem it solves, the packet or state that triggers it, the observable effect, the failure behavior, and the evidence that verifies it. Switch-internal names (ports, queues, tables) appear only after the concept they implement.

A. End-to-end walkthrough

A read enters the switch from the master. The program records the request's arrival time, forwards the request to the relay, and prepares to hold the relay's answer. When the acknowledgment and the response come back, the program holds each in a switch queue and releases them at fixed offsets from the recorded request time, so the master sees a constant

CLRT; on release it replicates the response and emits it as the two segments [28, 21]. A control command is handled on a second internal path: the SELECT prepares a hold, the OPERATE is admitted once and held, and it is released to the relay at an internal delay, while the acknowledgment and echo the master sees are placed at fixed offsets from the OPERATE's arrival. Two internal scheduling lanes keep the response path and the command path from interfering. A single terminal decision applies every forwarding effect.

B. Physical topology and packet paths

The master reaches the switch on one port and the switch reaches the SEL-751 on another; these are the only external cables. Three further ports are internal to the switch and carry no cable: two recirculation paths give the response treatment and the control treatment their own scheduling lanes, and one path is used by the switch's packet generator. The adversary observes only the master-facing port; the relay-facing port and the internal paths are not observed. Fig. 4 shows the detailed topology and names the internal ports and queues.

C. Recording the request time and arming deadlines

Problem. A held packet must be released at a fixed offset from the request, but the switch has no general-purpose timer. **Mechanism.** On the request (or the OPERATE), the program records an ingress timestamp T_0 and arms two deadlines, $T_0{+}A$ for the acknowledgment and $T_0{+}R$ for the response, from the policy offsets A and R; the master-visible CLRT is then $R - A$. Anchoring to T_0 rather than to the relay's later acknowledgment matters for the control command: because that acknowledgment exists only after the held OPERATE is released, anchoring to it would leak the hold. An offline model checks this anchoring against a mutant that arms the deadline at the acknowledgment, and the mutant is killed.

D. Reservoirs as data-plane timers

Problem. Realize a deadline from packet scheduling alone. **Mechanism.** A held packet waits in its queue behind a higher-priority reservoir of small "blocker" packets; a strict-priority scheduler drains the blockers first, so the held packet cannot leave until its reservoir empties, which the control plane sizes to the policy deadline. The switch's on-chip packet generator fills the reservoirs and never generates master or relay traffic. Two generator profiles are distinguished by a tag byte so that a read or select arm and a control hold trigger different reservoirs without ambiguity.

E. Response shaping: hold, release, replicate, carve

Response shaping normalizes the timing and shape of an eligible 49-byte response. The acknowledgment and the response are held in strict-priority queues and released at $T_0{+}A$ and $T_0{+}R$. On release the program steers the response into the switch's packet-replication engine, which produces two master-facing copies; egress then emits the 28-byte prefix from one copy and the 21-byte suffix from the other, advancing the suffix's TCP sequence number by 28 so the two segments abut. The relay produces the response; replication produces the two copies used for segmentation, and no clone fabricates the response. By design the program edits no DNP3 byte and recomputes no DNP3 CRC, updating only the IP and TCP checksums per segment; the hardware evidence bounds this to a sequence-contiguous, CRC-valid, checksum-valid 49-byte reconstruction, not byte identity to a source frame. The 49-byte frame is a 10-byte link header, an 18-byte first data block (16 data + 2 CRC), a second 18-byte block, and a 3-byte final block; byte 28 falls between the first and second blocks (Fig. 5). TCP may segment a byte stream at any offset, so cutting at byte 28 is a design and verification choice that keeps each segment aligned to a completed CRC block, not a TCP requirement.

F. The control command and its state

Problem. An outbound OPERATE exposes an operation-timing fingerprint, and releasing it on a fixed schedule would still reveal it. **Mechanism.** The SELECT prepares a hold by seeding the control reservoir, so the hold is armed before the OPERATE arrives. The switch admits one matching OPERATE, holds it, and releases it once to the relay at an internal delay drawn in the data plane from a bounded set. The master-visible acknowledgment and echo are released at $T_0{+}A$ and $T_0{+}R$, anchored to the OPERATE's own arrival and independent of the internal delay, so the observable gap carries no information about it. After release the transaction's generation is kept as a spent marker until the next SELECT; a retransmitted OPERATE that reads the spent marker is treated as a duplicate and dropped. **Evidence boundary.** The hardware captures establish the master-visible placement and the delay-invariant gap; the offline model verifies the single release; the relay-facing release time and the number of copies delivered are not captured, so single release is modeled, not measured.

G. Two independent scheduling lanes

Response shaping seeds acknowledgment and response reservoirs on high-priority queues. On one shared strict-priority schedule these reservoirs starve the control hold that would sit below them, shifting its release toward the response's release time instead of its own deadline (Fig. 6a). The design places the response queues on one internal recirculation lane and the control queues on a second (Fig. 6b), so each arbitrates on its own scheduler. A register-domain separation keeps a control-path packet from touching the response path's state. Both lanes are internal to the same switch and the same program.

H. One outcome, one commit

Earlier match-action stages compute one compact result and a single terminal table applies it. The terminal table is keyed on that result and is the only stage that sets the egress port, the queue, a drop, a bypass, or a multicast group; every defined result is mapped by a constant entry. The default action, taken only for the otherwise-unreachable unset result, forwards the

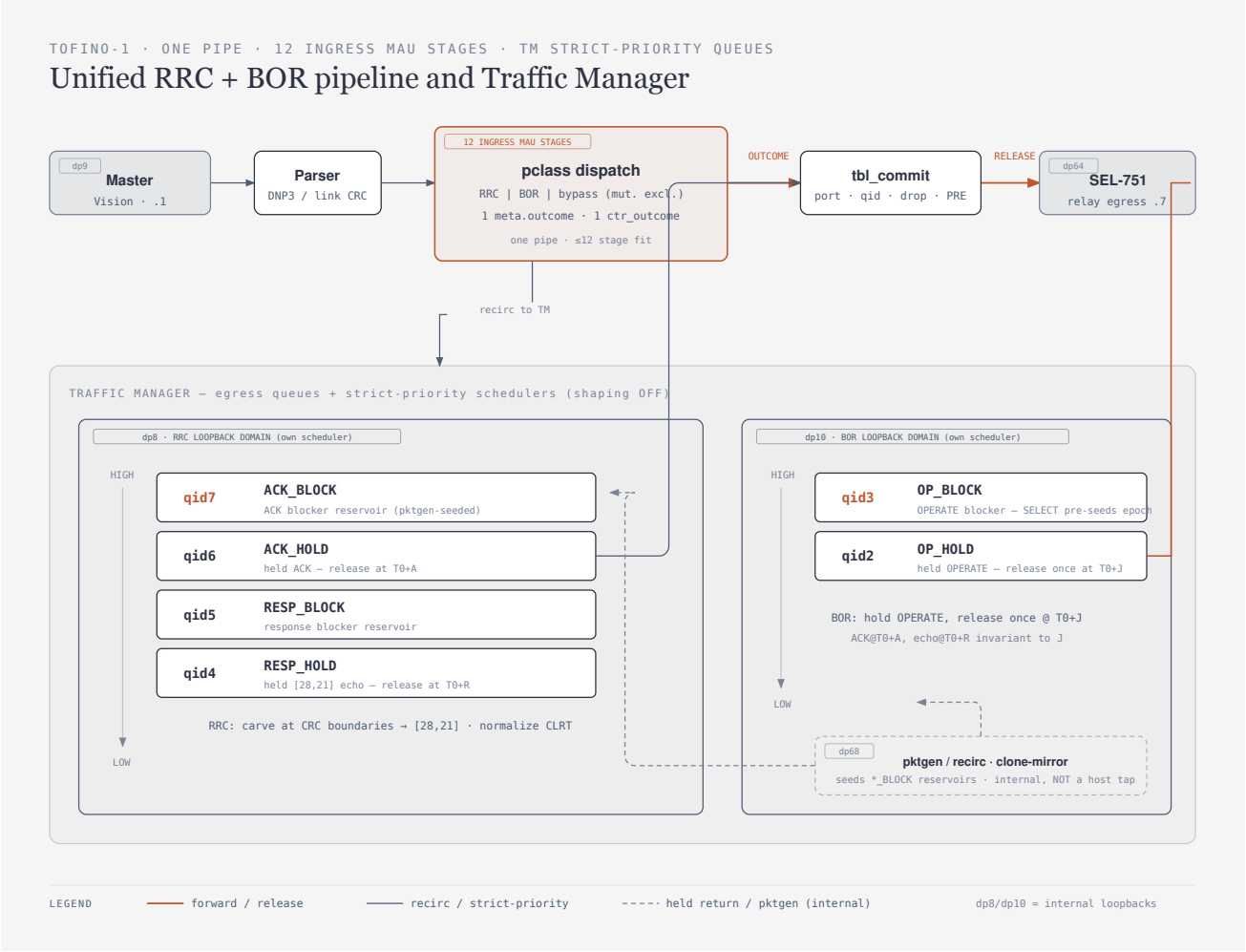

Fig. 4. Detailed switch pipeline. The parser and match-action stages compute one compact outcome that a single terminal table applies; the Traffic Manager holds the response reservoirs and queues on one internal lane (`dp8`) and the control queues on a second (`dp10`), with the packet generator (`dp68`) seeding the reservoirs; egress carves the eligible response to $[28, 21]$. Port and queue names appear only here.

packet unmodified rather than dropping it. This is distinct from the explicit dispositions of recognized traffic: internal-only packets such as a bad-port packet, a returning generator token, a clone, a duplicate response, or a retired blocker are mapped to an explicit drop, while a recognized but ineligible or unsupported response is mapped to an explicit forward. The single-table structure keeps the disposition auditable and, with a flattening of the decision logic into two mutually exclusive tables that co-place in one stage, lets the whole program fit one ingress pipeline within its stage budget.

I. Control plane, failure behavior, and safety

The control-plane layer brings up the ports, creates the queues at descending strict-priority levels with shaping disabled, installs the multicast group and the generator profiles, writes the timing parameters and the internal-delay set, and initializes the state. Every write is read back and compared, and a mismatch or a configuration warning leaves the mechanism disabled rather than partially configured; there is no configuration snapshot, and rollback is a forward, readback-checked teardown to a plain forwarding state. Traffic outside the eligible class, unsupported functions, and malformed frames are forwarded unchanged. The control-test harness that issues a physical SELECT and OPERATE is guarded: it permits only two isolated control points and refuses a breaker-close point. Across every guarded transaction all 32 relay outputs remained open and no breaker operation was attempted.

J. Resource fit and limitations

The program compiles for one Tofino-1 ingress pipeline within the twelve-stage match-action budget. The evaluation is on physical hardware, one switch and one relay; it is not a deployment study. The implementation does not observe the relay-facing link, does not normalize total response length, and is evaluated on one outstation and one eligible response class; Section V reports the measured consequences of these bounds. The loaded source and binary are identified by hash, the analysis regenerates deterministically from the captured traces, and a manifest verifies every derived artifact.

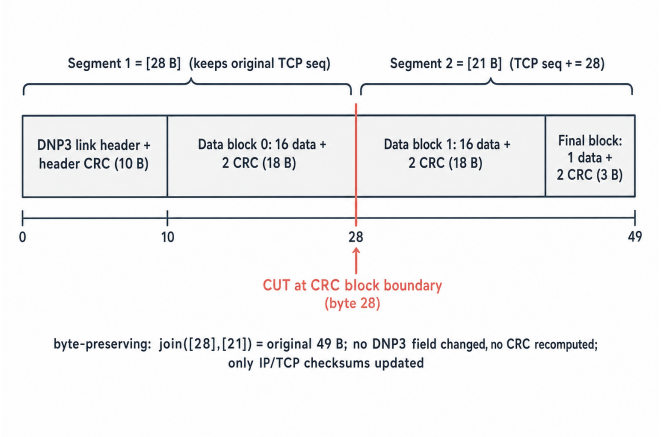

Fig. 5. The 49-byte response and the $[28, 21]$ carve. The cut at byte 28 is a boundary DNP3 already places between CRC-protected blocks; the prefix keeps the original TCP sequence number and the suffix advances it by 28. No DNP3 byte is edited and no DNP3 CRC is recomputed.

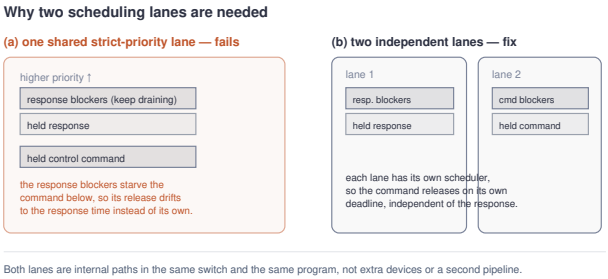

Fig. 6. Why two scheduling lanes are needed. (a) On one shared strict-priority schedule, the response's blocker packets starve the held control command below them, so its release drifts to the response's release time. (b) Two independent lanes each carry their own blockers and held packet, so the command releases on its own deadline. Both lanes are internal to the same switch and program.

V. Evaluation

We evaluate on a testbed with a physical master, one Tofino-1 switch, and a physical SEL-751 relay. Native and defended traffic are captured on the same loaded program, separated only by a runtime mode change, so the comparison isolates the defense. The defended captures carry no TCP timestamp option. The analysis regenerates deterministically from the traces, and a manifest verifies every derived artifact. Throughout, we separate what the hardware captures measure on the master-facing link from what the offline model verifies and what is not observed.

Timing (measured). The native CLRT median is 1.272 ms for read (standard deviation 1.354, n=999) and 2.107 ms for select (n=488), with heavy tails. The defended CLRT median is 4.001 ms for both classes, with standard deviation 0.022 and 0.021 ms, matching the policy $R - A = 4$ ms. The master-facing offsets A and R measure about 21 and 25 ms, which include a documented path and capture offset of roughly 1 ms over the configured 20 and 24 ms; the internal switch deadline was not instrumented separately.

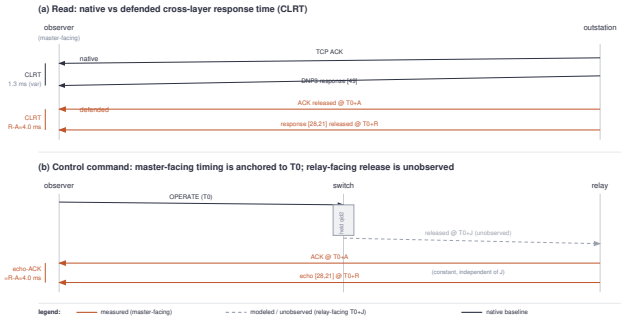

Fig. 7. Native versus defended timing, and the control-command timeline. The defended acknowledgment and response are placed at T_0+A and T_0+R; the internal release at T_0+J is on the relay-facing link and is not captured (dashed), so it is verified by the model rather than measured.

Shape and reconstruction (measured). The native read is one 49-byte segment. All 1,280 defended eligible responses (600 read, 500 select, and 90 select with 90 operate from the control transactions) present the $[28, 21]$ shape. Each is sequence-contiguous, valid under the DNP3 link and block CRCs, and valid under the IP and TCP checksums, and no defended response escapes as an unsplit 49-byte segment. Because no paired source-side capture exists, we report a CRC- and checksum-valid reconstruction, not byte identity to a source frame.

Control-command timing (measured and modeled). Fig. 7 shows the master-facing acknowledgment and echo of a control command. The echo-minus-acknowledgment gap is 4.00 ms and does not vary with the internal release delay: across the evaluated delays of 2, 6, and 12 ms the pooled gap median is 4.002 ms with standard deviation 0.026 ms (n=90). The relay-facing release time and the number of copies delivered to the relay are not captured; the offline model verifies the single release, and the captures establish that the master issued one OPERATE per transaction.

Fingerprint suppression (measured). We quantify the leak with the mutual information between the transaction class and the CLRT, using common bins and a permutation null. The native mutual information is 0.424 bits, well above the null band, and the defended value is 0.0018 bits, inside it. A logistic classifier trained to separate read from select by CLRT reaches 0.592 balanced accuracy on native traffic and 0.500, the chance baseline, on defended traffic. This is a transaction-class task on one device, not a device-identity task; the defense replaces the evaluated CLRT and segment-shape features of this SEL-751 and does not establish indistinguishability among devices.

Safety and resource fit (measured). The guarded control test permitted only two isolated control points and refused the breaker-close point; all 32 relay outputs remained open throughout, and no breaker operation was attempted. The program compiles for one ingress pipeline within the twelve-stage budget.

VI. Discussion

The defense replaces two master-facing features of one outstation with a fixed policy while preserving the protocol. It does not hide the total 49-byte length, which a full TCP reader recovers, and it does not establish indistinguishability among devices or resistance to every traffic-analysis feature. Extending the eligible class beyond the 49-byte responses would require a segment-shape policy for each response size; the CRC-block structure supports this, but we have not evaluated it. The strongest open item is relay-facing verification: a tap on the relay-facing link would turn the modeled single release into a measured property, and we leave that instrumentation to future work.

VII. Related Work

Device fingerprinting. Devices are identifiable from wire-side behavior without payload access: from remote clock skew [7], from timing signatures that reveal device and device type [8], and, for cyber-physical systems, from cross-layer response timing, physical-operation timing, and device physics [9], [18]. This line defines the threat we address rather than a defense. Our objective differs: we suppress the master-facing CLRT and segment-shape features these techniques exploit, for one evaluated device and its transaction classes.

Network-timing traffic analysis and its defenses. Inter-event timing leaks content, from keystroke intervals on SSH [19] to timing-only web-page recovery [6] and deep-learning website fingerprinting [5], though such accuracy is contested at realistic open-world scale [20]. Defenses shape the traffic: adaptive padding breaks the inter-packet pattern [11], comprehensive side-channel mitigation shapes a tenant's traffic to a leak-free schedule [21], and anonymity systems resist traffic analysis at the network layer [22], [23]. These target browser or cloud traffic and assume a cooperating end host or hypervisor that can add cover traffic. Our defense normalizes the acknowledgment-to-response timing of a protocol-bound DNP3 exchange in the network, with no endpoint change and no added application bytes, and fixes one cross-layer interval rather than obscuring a distribution.

Packet-size padding, morphing, and constant-rate shaping. Size defenses reshape a flow's length signature: morphing transforms one class's distribution into another's [10], dependent link padding bounds low-latency leakage [24], and constant-rate and burst-molding schemes fix the rate at a cost that is acceptable for one flow but prohibitive across a device fleet [12], [13]; later analysis shows per-packet padding and fragmentation still leave exploitable features [25]. The same size-and-rate channel identifies devices even under encryption, where shaping, made differentially private, is the countermeasure [26], [27]. These change or pad bytes and target the end host. Our mechanism neither pads nor rewrites DNP3 bytes; it re-segments the response only at existing CRC-block boundaries, so the reassembled message and every per-block CRC are preserved while the observed shape is fixed.

Reconnaissance, moving-target defense, and DNP3 security. A body of work disrupts grid reconnaissance rather than detecting its use: DefRec virtualizes physical functions to present deceptive content and connectivity [28], RAINCOAT randomizes data acquisition and injects decoys [29], and a companion testbed evaluates such approaches on grid cyber-physical infrastructure [30]; the reconnaissance these blunt ultimately enables false-data-injection and process-control attacks [3], [4], [31]. A parallel line detects rather than obfuscates, adapting specification-based and state-based intrusion detection to DNP3 and analyzing its attack surface [32]–[34], and in-network obfuscation has denied reconnaissance by rewriting data-plane topology against link-flooding attacks [17]. These operate at the content, connectivity, topology, or detection layer. Our defense instead obfuscates the wire-visible timing and shape of the transport exchange itself, below the semantics those defenses reshape, and complements rather than replaces them. Standards and incident analyses frame the setting [1], [2], [35].

Timing and shaping on programmable switches. Programmable data planes make in-network shaping practical. The P4 model exposes a programmable parser and match-action pipeline [36]; push-in-first-out queues define ideal programmable scheduling [37], and SP-PIFO approximates it with a few strict-priority queues on today's switches [38], the primitive class our deadline scheduler uses. Recent systems bring traffic-analysis defense fully into the data plane: Ditto pads and injects chaff to a fixed WAN pattern [14], Securitas obfuscates flows with flexible in-network shaping [15], Minos mounts a dynamic defense on programmable switches [16], and, closest to our setting, Hu and Lin enhance industrial-protocol security with programmable switches [39]. Our defense differs from these in two ways: it is byte-preserving and carves only at existing DNP3 boundaries, so the outstation's protocol remains valid, and it adds a control-command treatment that hides operation timing from the master-facing observer without exposing the hold.

VIII. Conclusion

An observer of unencrypted DNP3 traffic can fingerprint an outstation from the cross-layer response time and the segment shape of its responses, and neither encryption, endpoint changes, nor generic traffic-analysis defenses fit the constraints of a deployed DNP3 link. We placed the defense in the data plane of one programmable switch, where it holds the acknowledgment and response and releases them at fixed offsets from the request to normalize the response time, replicates and carves the eligible response into a fixed $[28, 21]$ shape at an existing CRC boundary, and holds a control command to pin its master-visible timing to the request. On a physical SEL-751 relay the defended cross-layer response time is a fixed 4.001 ms, every eligible response presents the $[28, 21]$ shape with a CRC- and checksum-valid reconstruction, the control-command gap stays at 4.00 ms across the evaluated internal delays, and a read-versus-select timing classifier falls to chance, within a single-device, master-facing evaluation boundary whose relay-facing verification we leave to future work.

REFERENCES

[1] IEEE, "IEEE standard for electric power systems communications—distributed network protocol (DNP3)," IEEE Std 1815-2012, 2012, iEEE Std 1815-2012.

[2] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016, tLP: White, 18 March 2016.

[3] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[4] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in *Proc. 6th ACM Symp. on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[6] S. Feghhi and D. J. Leith, "A web traffic analysis attack using only timing information," *IEEE Trans. on Information Forensics and Security*, vol. 11, no. 8, 2016.

[7] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," *IEEE Trans. on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[8] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[9] D. Formby, P. Srinivasan, A. M. Leonard, J. D. Rogers, and R. A. Beyah, "Who's in control of your control system? Device fingerprinting for cyber-physical systems," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[10] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2009.

[11] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in *Proc. European Symp. on Research in Computer Security (ESORICS)*, 2016, pp. 27–46.

[12] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[13] T. Wang and I. Goldberg, "Walkie-talkie: An efficient defense against passive website fingerprinting attacks," in *Proc. 26th USENIX Security Symposium*, 2017, pp. 1375–1390.

[14] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN traffic obfuscation at line rate," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2022.

[15] G. Xie *et al.*, "Securitas: Defending against traffic analysis attacks with flexible in-network obfuscation," in *Proc. 23rd USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2026.

[16] Z. Wang *et al.*, "Minos: A lightweight and dynamic defense against traffic analysis in programmable data planes," in *Proc. USENIX Annual Technical Conference (ATC)*, 2025.

[17] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and practical network topology obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[18] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for cyber-physical system security: Device physics matters too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, 2018.

[19] D. X. Song, D. Wagner, and X. Tian, "Timing analysis of keystrokes and timing attacks on SSH," in *Proc. 10th USENIX Security Symposium*, 2001.

[20] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website fingerprinting at internet scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[21] A. Mehta, M. Alzayat, R. De Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive network side-channel mitigation in the cloud," in *Proc. 31st USENIX Security Symposium*, 2022.

[22] C. Chen, D. E. Asoni, D. Barrera, G. Danezis, and A. Perrig, "HORNET: High-speed onion routing at the network layer," in *Proc. 22nd ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2015.

[23] C. Chen, D. E. Asoni, A. Perrig, D. Barrera, G. Danezis, and C. Troncoso, "TARANET: Traffic-analysis resistant anonymity at the network layer," in *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*, 2018.

[24] W. Wang, M. Motani, and V. Srinivasan, "Dependent link padding algorithms for low latency anonymity systems," in *Proc. 15th ACM Conf. on Computer and Communications Security (CCS)*, 2008.

[25] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, I still see you: Why efficient traffic analysis countermeasures fail," in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2012, pp. 332–346.

[26] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, vol. 2019, no. 3, pp. 128–148, 2019.

[27] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A differentially private network side-channel mitigation system," in *Proc. 33rd USENIX Security Symposium*, 2024.

[28] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing physical function virtualization to disrupt reconnaissance of power grids' cyber-physical infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[29] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of network communication in power grid cyber infrastructure to mislead attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[30] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-physical testbed: Case study to evaluate anti-reconnaissance approaches on power grids' cyber-physical infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[31] S. Sridhar, A. Hahn, and M. Govindarasu, "Cyber-physical system security for the electric power grid," *Proc. of the IEEE*, vol. 100, no. 1, pp. 210–224, 2012.

[32] H. Lin, A. J. Slagell, C. Di Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a specification-based intrusion detection system for the DNP3 protocol," in *Proc. 8th Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.

[33] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 state-based intrusion detection system," in *Proc. 24th IEEE Int. Conf. on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.

[34] S. East, J. Butts, M. Papa, and S. Shenoi, "A taxonomy of attacks on the DNP3 protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol. 311)*. Springer, 2009, pp. 67–81.

[35] K. Stouffer, V. Pillitteri, S. Lightman, M. Abrams, and A. Hahn, "Guide to industrial control systems (ICS) security," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-82 Rev. 2, 2015.

[36] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[37] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, A. Agrawal, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable packet scheduling at line rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[38] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating push-in first-out behaviors using strict-priority queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[39] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial network protocol security enhancement using programmable switches," in *Proc. IEEE Int. Conf. on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.